

P P1. Regresión

Objetivo del proyecto:

El objetivo principal es predecir la longitud total de los tlacuaches (totlngth) a partir de variables morfológicas y demográficas, como edad, sexo, tamaño de cola, sitio de captura, entre otras. Se busca identificar cuáles de estas variables tienen mayor poder predictivo y caracterizar la relación entre ellas y la longitud total, utilizando modelos de regresión lineal y no lineal. Esto permitirá entender mejor los factores que influyen en el tamaño de los tlacuaches y evaluar la capacidad de los modelos para realizar predicciones precisas y generalizables.

Datos obtenidos de <https://www.kaggle.com/datasets/abrambeyer/openintro-possum>

- case: caso de observación
- site: El sitio donde el tlacuache fue atrapado.
- Pop: Población, Vic (Victoria) o other (New South Wales o Queensland).
- sex: m (macho) o f (hembra).
- age: Edad.
- hdlngh: Longitud de cabeza en mm.
- skullw: Ancho de cráneo en mm.
- totlngth: Longitud total en cm (variable respuesta de interés).
- taill: Tamaño de cola en cm.
- footlgh: Longitud de pies en mm.
- earconch: Longitud de caracola de la oreja en mm.
- eye: Distancia del canto medial al canto lateral del ojo derecho en mm.
- chest: Circunferencia del pecho en cm.
- belly: Circunferencia del vientre en cm.

Fuente de los datos:

Los datos fueron descargados de [Kaggle: OpenIntro Possum Dataset](#).

```
In [56]: import pandas as pd

df = pd.read_csv('possum.csv')

# Cantidad de observaciones y variables
print("Cantidad de observaciones:", df.shape[0])
print("Cantidad de variables:", df.shape[1])

# Nombres de las variables
print("\nNombres de las variables:")
print(df.columns.tolist())
```

```
# Tipos de variables
print("\nTipos de variables:")
print(df.dtypes)

# Primeras filas del dataframe
print("\nPrimeras filas del dataframe:")
print(df.head())

# Resumen estadístico de las variables numéricas
print("\nResumen estadístico de las variables numéricas:")
print(df.describe())

# Distribución de la variable respuesta de interés (sex)
import matplotlib.pyplot as plt

df['sex'].value_counts().plot(kind='bar')
plt.title('Distribución de la variable respuesta: sex')
plt.xlabel('Sexo')
plt.ylabel('Cantidad')
plt.show()
```

Cantidad de observaciones: 104
Cantidad de variables: 14

Nombres de las variables:
['case', 'site', 'Pop', 'sex', 'age', 'hdlngth', 'skullw', 'totlngth', 'taill', 'footlngth', 'earconch', 'eye', 'chest', 'belly']

Tipos de variables:
case int64
site int64
Pop object
sex object
age float64
hdlngth float64
skullw float64
totlngth float64
taill float64
footlngth float64
earconch float64
eye float64
chest float64
belly float64
dtype: object

Primeras filas del dataframe:

	case	site	Pop	sex	age	hdlngth	skullw	totlngth	taill	footlngth	\
0	1	1	Vic	m	8.0	94.1	60.4	89.0	36.0	74.5	
1	2	1	Vic	f	6.0	92.5	57.6	91.5	36.5	72.5	
2	3	1	Vic	f	6.0	94.0	60.0	95.5	39.0	75.4	
3	4	1	Vic	f	6.0	93.2	57.1	92.0	38.0	76.1	
4	5	1	Vic	f	2.0	91.5	56.3	85.5	36.0	71.0	

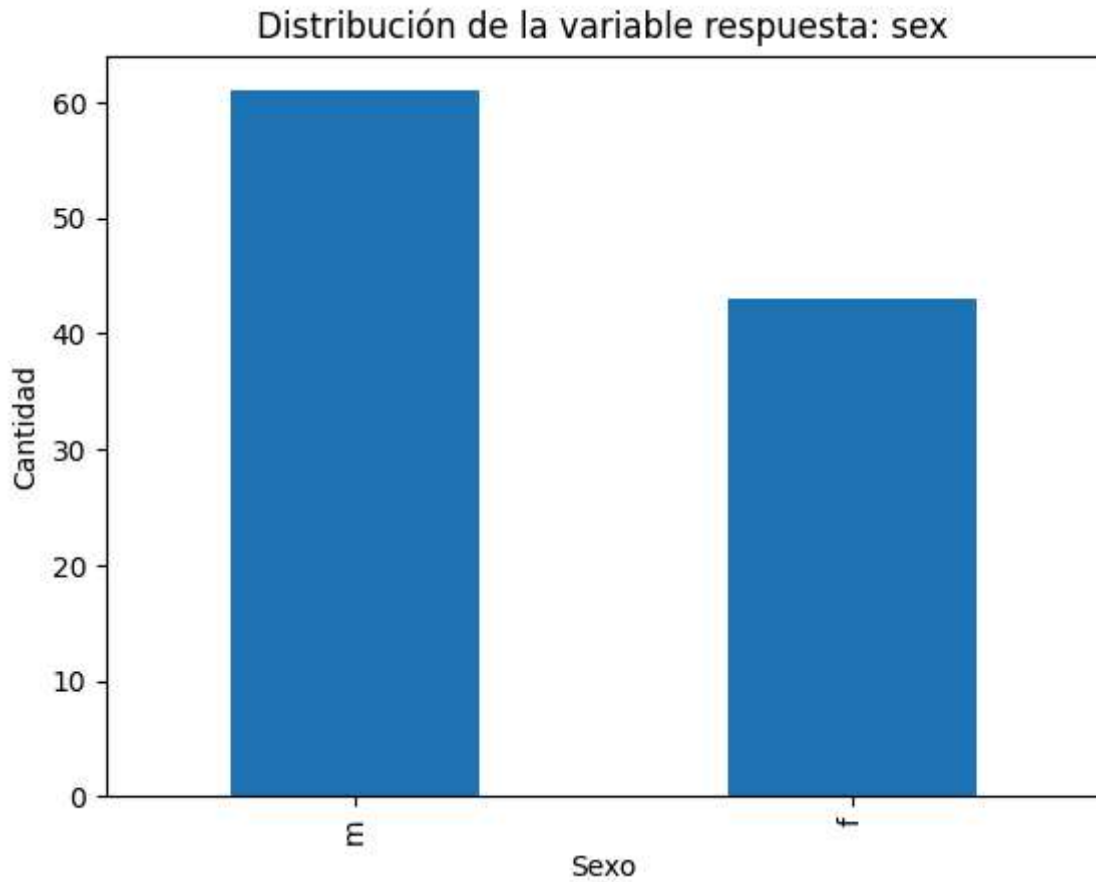
	earconch	eye	chest	belly
0	54.5	15.2	28.0	36.0
1	51.2	16.0	28.5	33.0
2	51.9	15.5	30.0	34.0
3	52.2	15.2	28.0	34.0
4	53.2	15.1	28.5	33.0

Resumen estadístico de las variables numéricas:

	case	site	age	hdlngth	skullw	totlngth	\
count	104.000000	104.000000	102.000000	104.000000	104.000000	104.000000	
mean	52.500000	3.625000	3.833333	92.602885	56.883654	87.088462	
std	30.166206	2.349086	1.909244	3.573349	3.113426	4.310549	
min	1.000000	1.000000	1.000000	82.500000	50.000000	75.000000	
25%	26.750000	1.000000	2.250000	90.675000	54.975000	84.000000	
50%	52.500000	3.000000	3.000000	92.800000	56.350000	88.000000	
75%	78.250000	6.000000	5.000000	94.725000	58.100000	90.000000	
max	104.000000	7.000000	9.000000	103.100000	68.600000	96.500000	

	taill	footlngth	earconch	eye	chest	belly
count	104.000000	103.000000	104.000000	104.000000	104.000000	104.000000
mean	37.009615	68.459223	48.130769	15.046154	27.000000	32.586538
std	1.959518	4.395306	4.109380	1.050374	2.045597	2.761949
min	32.000000	60.300000	40.300000	12.800000	22.000000	25.000000
25%	35.875000	64.600000	44.800000	14.400000	25.500000	31.000000

50%	37.000000	68.000000	46.800000	14.900000	27.000000	32.500000
75%	38.000000	72.500000	52.000000	15.725000	28.000000	34.125000
max	43.000000	77.900000	56.200000	17.800000	32.000000	40.000000



Cambiamos las variables categóricas a numéricas para que se puedan utilizar en un modelo de regresión lineal. Como la variable de sexo es categórica, haremos una columna dummy con 0 si es hembra y 1 si es macho. La variable site tiene dos categorías, Vic y other. Haremos una columna dummy con 1 si es Vic y 0 si es other.

```
In [57]: df['male'] = df['sex'].map({'f': 0, 'm': 1})
df = df.drop(columns=['sex'])

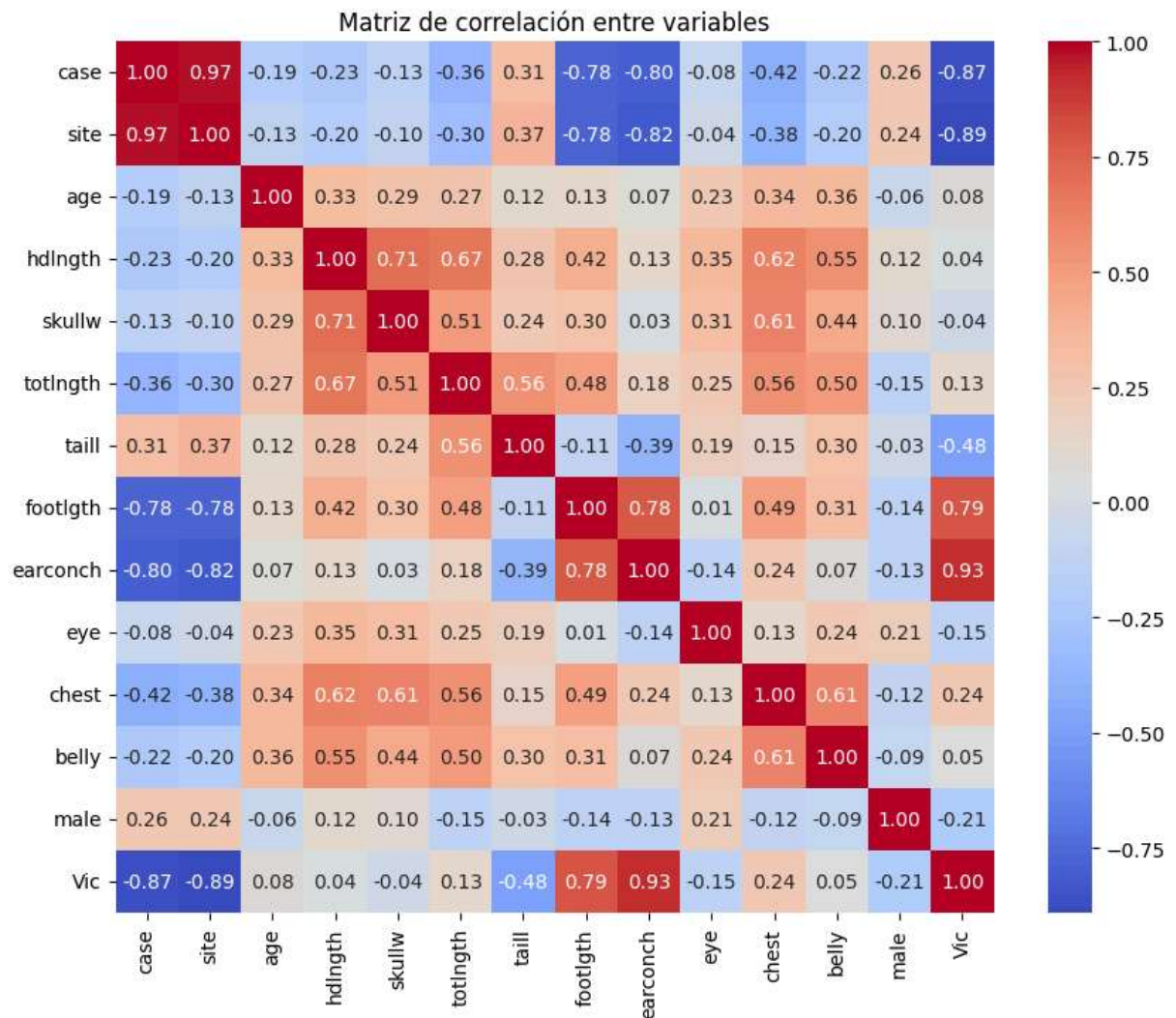
df['Vic'] = df['Pop'].map({'Vic': 1, 'other': 0})
df = df.drop(columns=['Pop'])
```

Verificamos que ya no haya datos faltantes

```
In [58]: # Checamos huecos en los datos
df.isnull().sum()
# Eliminamos las filas con datos faltantes ya que son pocos
df = df.dropna()
```

Verificamos la colinealidad entre variables predictoras usando un mapa de calor de la matriz de correlación para identificar posibles problemas de multicolinealidad.

```
In [59]: import seaborn as sns
plt.figure(figsize=(10, 8))
sns.heatmap(df.corr(), annot=True, fmt=".2f", cmap='coolwarm')
plt.title('Matriz de correlación entre variables')
plt.show()
df = df.drop(columns=['case', 'Vic'])
```



La alta colinealidad entre site y vic nos sugiere eliminar dos de las tres variables:

- Eliminaremos case ya que es un identificador único y no aporta información predictiva.
- Eliminaremos Vic ya que site aporta más información.

La metodología de selección hacia atrás fue elegida porque tenemos una cantidad alta de variables y buscamos eliminar las que no aportan al modelo, optimizando la predicción del modelo. Este método es útil cuando se sospecha que algunas variables pueden ser redundantes o no relevantes, permitiendo identificar el subconjunto más informativo.

Como resultado, se eliminaron variables que no mejoraban el R2 ajustado del modelo, quedando solo aquellas que contribuyen de manera significativa a la predicción de la

longitud total. Esto ayuda a evitar el sobreajuste y mejora la interpretabilidad del modelo, asegurando que solo las variables más relevantes sean utilizadas.

```
In [60]: from sklearn.linear_model import LinearRegression
import numpy as np

def adjusted_r2_score(model, X, y):
    r2 = model.score(X, y)
    n = X.shape[0]
    p = X.shape[1]
    return 1 - (1 - r2) * (n - 1) / (n - p - 1)

features = list(X.columns)
current_X = X.copy()
best_adj_r2 = -np.inf
improvement = True

while improvement and len(features) > 1:
    scores = []
    for feature in features:
        temp_features = [f for f in features if f != feature]
        model = LinearRegression().fit(current_X[temp_features], y)
        adj_r2 = adjusted_r2_score(model, current_X[temp_features], y)
        scores.append((adj_r2, feature))
    scores.sort(reverse=True)
    best_score, feature_to_remove = scores[0]
    if best_score > best_adj_r2:
        best_adj_r2 = best_score
        features.remove(feature_to_remove)
        print(f"Atributo eliminado: {feature_to_remove}, R^2 ajustado: {best_score}")
    else:
        improvement = False

print("Características finales seleccionadas:", features)
```

```
Atributo eliminado: age, R^2 ajustado: 0.7498
Atributo eliminado: eye, R^2 ajustado: 0.7523
Atributo eliminado: skullw, R^2 ajustado: 0.7548
Atributo eliminado: belly, R^2 ajustado: 0.7570
Atributo eliminado: chest, R^2 ajustado: 0.7589
Atributo eliminado: earconch, R^2 ajustado: 0.7603
Características finales seleccionadas: ['site', 'hdlngth', 'taill', 'footlght', 'male']
```

Solo las siguientes variables fueron seleccionadas tras el proceso de selección de características:

- site
- hdlngth
- taill
- footlght
- male

Estas variables mostraron mayor poder predictivo y contribuyeron significativamente al modelo de regresión para predecir la longitud total (totlngth).

Con esto nos podemos dar cuenta que aunque la edad no afecta mucho el tamaño total de estos tlacuaches, pero su sexo sí.

Ajusté un modelo de regresión lineal con las variables seleccionadas, ya que la relación entre las variables predictoras y la respuesta es principalmente lineal según el análisis.

Entrené un modelo de regresión polinomial de grado 2 para verificar si una relación no lineal mejoraba el ajuste, comparando los valores de R².

El R² del modelo lineal fue 0.77 y el del modelo polinomial 0.84, mostrando que el modelo polinomial captura mejor la variabilidad, aunque el modelo lineal es más interpretable.

Coeficientes estimados del modelo lineal: [-0.71185787 0.47662833 1.30270388 0.07828891 -0.49589922] Intercepto: -8.274318176382081

Un coeficiente es significativo si su valor absoluto es considerable y consistente con el contexto del problema, y si el modelo mejora al incluirlo. Por ejemplo, los coeficientes de (0.47) y (1.30) son relativamente altos, lo que indica una fuerte asociación positiva con la longitud total.

Para afirmar estadísticamente la significancia, sería necesario calcular los intervalos de confianza o los valores p, pero el proceso de selección y el tamaño de los coeficientes ya nos dan una buena indicación de su relevancia en el modelo.

```
In [61]: from sklearn.model_selection import train_test_split
from sklearn.preprocessing import PolynomialFeatures
from sklearn.pipeline import Pipeline

df = df[features + ['totlngth']]

X = df.drop(columns=['totlngth'])
y = df['totlngth']

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_sta

# Entrenar modelo lineal con datos de entrenamiento
linear_model = LinearRegression()
linear_model.fit(X_train, y_train)
print("Coeficientes del modelo lineal:", linear_model.coef_)
print("Intercepto:", linear_model.intercept_)
print("R^2 del modelo lineal (train):", linear_model.score(X_train, y_train))
print("R^2 del modelo lineal (test):", linear_model.score(X_test, y_test))

# Modelo de regresión polinomial (grado 2)
poly_model = Pipeline([
    ('polynomialfeatures', PolynomialFeatures(degree=2, include_bias=False)),
    ('linearregression', LinearRegression())
])
```



```

poly_model.fit(X_train, y_train)
print("R^2 del modelo polinomial (grado 2, train):", poly_model.score(X_train, y_train))
print("R^2 del modelo polinomial (grado 2, test):", poly_model.score(X_test, y_test))

# Graficar valores reales vs predichos en el set de prueba para ambos modelos
y_test_pred_linear = linear_model.predict(X_test)
y_test_pred_poly = poly_model.predict(X_test)

plt.figure(figsize=(8, 5))
plt.scatter(y_test, y_test_pred_linear, label='Lineal', color='blue', alpha=0.7)
plt.scatter(y_test, y_test_pred_poly, label='Polinomial grado 2', color='red', alpha=0.7)
plt.plot([y_test.min(), y_test.max()], [y_test.min(), y_test.max()], 'k--', lw=2)
plt.xlabel('Valores reales')
plt.ylabel('Predicciones')
plt.title('Valores reales vs predichos (test)')
plt.legend()
plt.show()

```

Coefficientes del modelo lineal: [-0.71185787 0.47662833 1.30270388 0.07828891 -0.49589922]

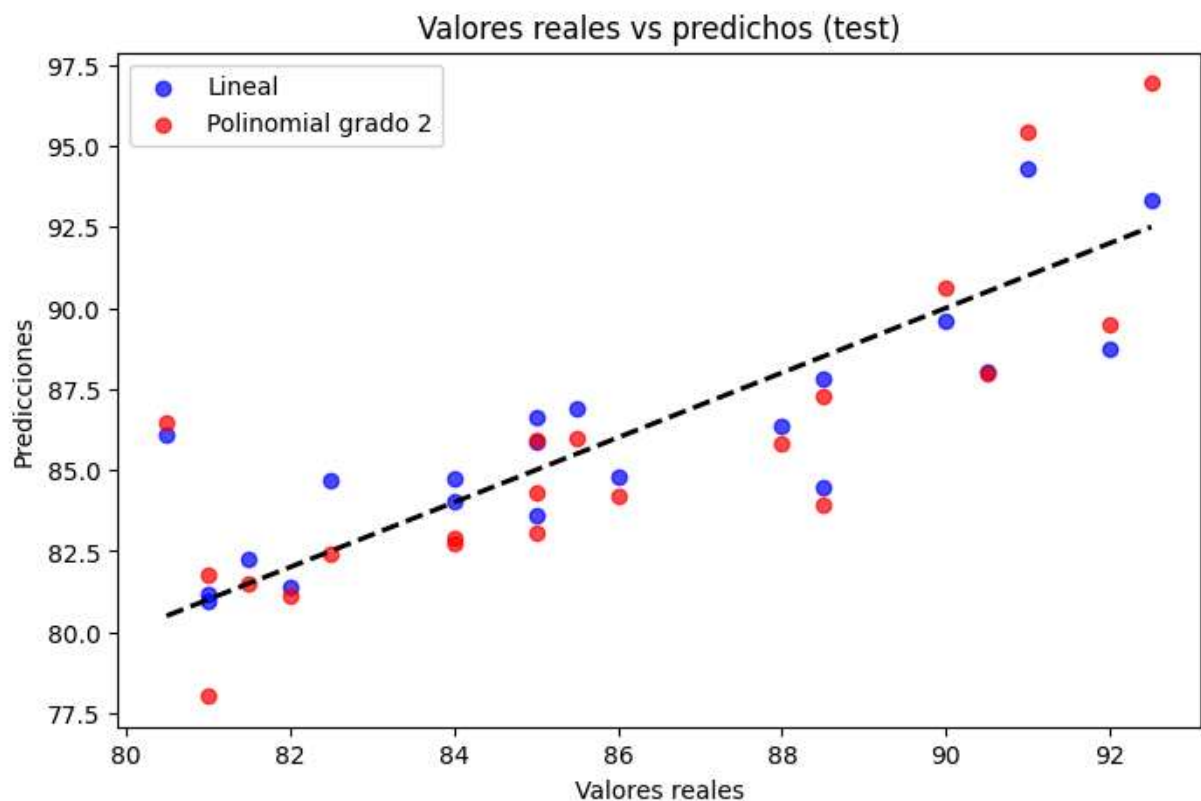
Intercepto: -7.626044096800484

R² del modelo lineal (train): 0.7815101439455625

R² del modelo lineal (test): 0.6806403026653536

R² del modelo polinomial (grado 2, train): 0.8564164391677656

R² del modelo polinomial (grado 2, test): 0.5376934288333421



Interpretación de las métricas

El MSE de train y test nos indica el promedio de las diferencias al cuadrado entre los valores reales y los predichos. Un valor más bajo indica mejores predicciones. El modelo lineal tiene un MSE de test 4.85 mientras que el modelo polinomial tiene un MSE de test mucho mayor

(35.61), indicando que el modelo lineal generaliza mejor a datos no vistos. El R^2 del modelo lineal en test es 0.656, mostrando que explica el 65.6% de la variabilidad en los datos de prueba, mientras que el modelo polinomial tiene un R^2 negativo en test, sugiriendo un mal ajuste.

```
In [ ]: from sklearn.metrics import mean_squared_error, r2_score

# Calculamos el error cuadrático medio (MSE) en el set de prueba
mse = mean_squared_error(y_test, y_test_pred_linear)
mse_train = mean_squared_error(y_train, linear_model.predict(X_train))
print("Error cuadrático medio (MSE) en test:", mse)
print("Error cuadrático medio (MSE) en train:", mse_train)

# Calculamos el coeficiente de determinación  $R^2$  en el set de prueba
r2 = r2_score(y_test, y_test_pred_linear)
r2_train = r2_score(y_train, linear_model.predict(X_train))
print("Coeficiente de determinación  $R^2$  en test:", r2)
print("Coeficiente de determinación  $R^2$  en train:", r2_train)

# Modelo polinomial
mse_poly = mean_squared_error(y_test, y_test_pred_poly)
mse_poly_train = mean_squared_error(y_train, poly_model.predict(X_train))
r2_poly = r2_score(y_test, y_test_pred_poly)
r2_poly_train = r2_score(y_train, poly_model.predict(X_train))
print("Error cuadrático medio (MSE) en test (polinomial):", mse_poly)
print("Error cuadrático medio (MSE) en train (polinomial):", mse_poly_train)
print("Coeficiente de determinación  $R^2$  en test (polinomial):", r2_poly)
print("Coeficiente de determinación  $R^2$  en train (polinomial):", r2_poly_train)
```

```
Error cuadrático medio (MSE) en test: 4.851403512181315
Error cuadrático medio (MSE) en train: 3.7589862117932453
Coeficiente de determinación  $R^2$  en test: 0.6555909612247327
Coeficiente de determinación  $R^2$  en train: 0.7876488178294161
Error cuadrático medio (MSE) en test (polinomial): 35.607980136421034
Error cuadrático medio (MSE) en train (polinomial): 0.030451431036759725
Coeficiente de determinación  $R^2$  en test (polinomial): -1.5278685190215193
Coeficiente de determinación  $R^2$  en train (polinomial): 0.9982797496412318
```

El coeficiente de 'hdlngh' está entre 0.321 y 0.632 con un 95% de confianza, lo que indica que, manteniendo constantes las demás variables, un aumento de 1 mm en la longitud de la cabeza se asocia con un incremento estimado de entre 0.32 y 0.63 cm en la longitud total del tlacuache.

Además, la longitud total promedio estimada para un tlacuache con valores promedio en las variables predictoras es de 87.63 cm, con un intervalo de confianza del 95% entre 87.17 y 88.08 cm.

```
In [62]: import statsmodels.api as sm

# Usamos las variables seleccionadas para inferencia
X_infer = X_train.copy()
X_infer = sm.add_constant(X_infer) # Agrega el intercepto
y_infer = y_train
```

```
# Ajustar el modelo de regresión lineal con statsmodels
model_infer = sm.OLS(y_infer, X_infer).fit()

# Resumen del modelo con intervalos de confianza
print(model_infer.summary())

# Inferencia para intervalo de confianza para el coeficiente de 'hdlngh'
conf_int = model_infer.conf_int().loc['hdlngh']
print(f"\nEl coeficiente de 'hdlngh' está entre {conf_int[0]:.3f} y {conf_int[1]:.3f}")

# Inferencia sobre la media de la respuesta para una observación promedio
mean_pred = model_infer.get_prediction(X_infer.mean().to_frame().T)
mean_ci = mean_pred.conf_int()[0]
print(f"\nLa longitud total promedio estimada es {mean_pred.predicted_mean[0]:.2f}")
```

OLS Regression Results

```
=====
Dep. Variable:          totlngh      R-squared:                0.782
Model:                  OLS          Adj. R-squared:           0.767
Method:                 Least Squares  F-statistic:              52.94
Date:                  Mon, 08 Sep 2025  Prob (F-statistic):       4.50e-23
Time:                  00:00:54      Log-Likelihood:           -167.62
No. Observations:      80           AIC:                     347.2
Df Residuals:          74           BIC:                     361.5
Df Model:              5
Covariance Type:       nonrobust
=====
```

	coef	std err	t	P> t	[0.025	0.975]
const	-7.6260	7.507	-1.016	0.313	-22.584	7.332
site	-0.7119	0.187	-3.804	0.000	-1.085	-0.339
hdlngh	0.4766	0.078	6.097	0.000	0.321	0.632
taill	1.3027	0.142	9.156	0.000	1.019	1.586
footlgth	0.0783	0.095	0.824	0.413	-0.111	0.268
male	-0.4959	0.494	-1.003	0.319	-1.481	0.489

```
=====
Omnibus:                5.499      Durbin-Watson:           1.974
Prob(Omnibus):          0.064      Jarque-Bera (JB):        4.767
Skew:                  -0.565      Prob(JB):               0.0922
Kurtosis:              3.392      Cond. No.                3.98e+03
=====
```

Notes:

[1] Standard Errors assume that the covariance matrix of the errors is correctly specified.

[2] The condition number is large, 3.98e+03. This might indicate that there are strong multicollinearity or other numerical problems.

El coeficiente de 'hdlngh' está entre 0.321 y 0.632 con 95% de confianza.

La longitud total promedio estimada es 87.63 cm, con un intervalo de confianza del 95% entre 87.17 y 88.08.